

# Zealous System Technical Test Prep

2027 Batch Campus Placement Drive

Vishal Chudasama · MCA, LJ University, Ahmedabad

Applying for: **React JS Developer**

|                          |                                                                 |
|--------------------------|-----------------------------------------------------------------|
| <b>Company</b>           | Zealous System Pvt. Ltd.                                        |
| <b>Role</b>              | React JS Developer                                              |
| <b>Batch</b>             | 2027                                                            |
| <b>Scope of this set</b> | Technical MCQs only — no aptitude, HR, or behavioural questions |

This document contains 30 technical multiple-choice questions (15 Common Technical + 15 React JS) for revision before the Zealous System technical test. Each question includes 4 options, the correct answer, and an explanation. Difficulty is mixed — mostly easy/medium with a few harder, interview-style questions — suitable for a fresher/junior React JS role.

# Contents

|                  |                                                                                 |              |
|------------------|---------------------------------------------------------------------------------|--------------|
| <b>Section 1</b> | Common Technical (Programming, OOP, JS, HTML/CSS, HTTP, Git, SQL, Web Concepts) | 15 Questions |
| <b>Section 2</b> | React JS (Components, Hooks, Routing, Context, Performance, and more)           | 15 Questions |

## Section 1 · Common Technical (15 Questions)

Fundamentals that a fresher/junior developer is commonly tested on, regardless of the specific role — covering programming basics, OOP, JavaScript, HTML/CSS, HTTP/REST, Git, SQL, and general web concepts.

Programming Fundamentals Easy

C1.

**What best describes how JavaScript code is executed?**

- (A) JavaScript must be fully compiled into a standalone executable before it can run
- (B) JavaScript is purely interpreted line-by-line with no compilation involved at any stage
- (C) A JavaScript engine (like V8) parses and executes the code, using Just-In-Time (JIT) compilation internally to improve performance
- (D) JavaScript requires a separate manual compile step before it can run in a browser or Node.js

**Correct Answer: (C)**

**Explanation:** There's no separate "compile" step a developer runs manually. A JS engine like V8 parses the source and uses JIT (Just-In-Time) compilation internally — interpreting and compiling hot code paths on the fly — which is why JS is often loosely called "interpreted" even though real compilation happens under the hood.

OOP Easy

C2.

**Which four concepts form the core principles of Object-Oriented Programming?**

- (A) Encapsulation, Abstraction, Inheritance, Polymorphism
- (B) Modularity, Recursion, Iteration, Compilation
- (C) Variables, Functions, Loops, Conditionals
- (D) Classes, Objects, Methods, Attributes

**Correct Answer: (A)**

**Explanation:** Encapsulation (bundling data with the methods that act on it), Abstraction (hiding implementation detail), Inheritance (reusing/extending a parent class), and Polymorphism (the same operation behaving differently depending on the object) are the four pillars of OOP.

OOP Medium

C3.

**What is the difference between method overloading and method overriding?**

- (A) Overloading = multiple methods with the same name but different parameters in the same class (resolved at compile time); Overriding = a subclass redefining a parent method with the same signature (resolved at runtime)
- (B) Overloading only happens in a subclass; Overriding only happens within the same class
- (C) Both terms describe exactly the same concept
- (D) Overloading changes only the return type; Overriding changes only the parameter types

**Correct Answer: (A)**

**Explanation:** Overloading is compile-time (static) polymorphism within one class; overriding is runtime (dynamic) polymorphism between a parent and child class, where the child provides its own implementation of an inherited method.

JavaScript Easy

C4.

**Which statement correctly describes var, let, and const in JavaScript?**

- (A) var is block-scoped, while let and const are function-scoped
- (B) let and const are block-scoped; var is function-scoped. let allows reassignment, const does not
- (C) var, let, and const all behave identically in terms of scope and reassignment
- (D) Variables declared with const can never hold objects or arrays

**Correct Answer: (B)**

**Explanation:** var is function-scoped (and hoisted), while let/const are block-scoped. let can be reassigned; const cannot be reassigned, though an object or array it holds can still have its internal contents mutated.

JavaScript

Medium

C5.

**What is a closure in JavaScript?**

- (A) A function that takes no parameters
- (B) A function bundled together with references to its surrounding (lexical) scope, allowing it to access variables from an outer function even after that outer function has returned
- (C) A built-in JavaScript error type thrown when a variable is undefined
- (D) A feature that only works inside arrow functions

**Correct Answer: (B)**

**Explanation:** Closures let an inner function "remember" variables from its enclosing scope, which is commonly used for data privacy (private variables) and for callbacks/event handlers that need to retain state between calls.

JavaScript

Easy

C6.

**What is the difference between == and === in JavaScript?**

- (A) == checks both value and type; === checks value only
- (B) === checks both value and type (strict equality); == converts operands to the same type before comparing (loose equality)
- (C) Both operators behave identically in all cases
- (D) == can only be used to compare strings

**Correct Answer: (B)**

**Explanation:** === (strict equality) compares value and type with no conversion. == (loose equality) first coerces operands to a common type, which can cause surprising results (e.g., "5" == 5 is true, but "5" === 5 is false).

JavaScript

Medium

C7.

**How do Promises and async/await relate to each other in JavaScript?**

- (A) A Promise represents a value that may be available now, in the future, or never (pending/fulfilled/rejected); async/await is syntactic sugar that lets promise-based code be written in a more synchronous-looking style
- (B) Promises and async/await are completely unrelated JavaScript features
- (C) async/await removes the need for the JavaScript event loop entirely
- (D) A Promise can only be used together with setTimeout

**Correct Answer: (A)**

**Explanation:** `async` functions always return a Promise, and `await` pauses execution within that function until the awaited Promise settles — making asynchronous code easier to read than chained `.then()` calls, while still running on the same underlying event loop and microtask queue.

HTML

Easy

C8.

**What is "semantic HTML," and why does it matter?**

- (A) HTML tags that only affect visual styling, such as `<div>` and `<span>`
- (B) HTML elements that clearly describe their meaning/purpose in the markup — such as `<header>`, `<nav>`, `<article>`, and `<footer>` — which improves accessibility (screen readers) and SEO
- (C) A deprecated version of HTML no longer supported by modern browsers
- (D) Tags that can only be used inside a `<table>` element

**Correct Answer: (B)**

**Explanation:** Semantic tags convey meaning about their content's role in the page, which helps assistive technologies, search engines, and other developers understand the structure — unlike generic `<div>/<span>` tags, which carry no inherent meaning.

CSS Easy

C9.

What is the correct order of layers in the CSS box model, from innermost to outermost?

- (A) Content → Padding → Border → Margin
- (B) Margin → Border → Padding → Content
- (C) Only Content and Border exist in the box model
- (D) Padding always sits outside the border

Correct Answer: (A)

**Explanation:** Every box is built as Content (the actual text/image), then Padding (space inside the border), then Border, then Margin (space outside the border, separating it from other elements).

CSS Medium

C10.

Which statement about the CSS "position" property is correct?

- (A) "relative" removes an element from the normal document flow entirely
- (B) "absolute" positions an element relative to its nearest positioned ancestor (or the initial containing block if none exists), removing it from the normal flow
- (C) "fixed" scrolls along with the page content
- (D) "sticky" behaves exactly like "static" at all times

Correct Answer: (B)

**Explanation:** `relative` stays in normal flow but can be nudged with offsets; `absolute` is removed from flow and positioned against the nearest ancestor with a non-static position; `fixed` is positioned relative to the viewport and stays in place while scrolling; `sticky` toggles between relative and fixed behavior based on scroll position.

HTTP / REST Medium

C11.

What is the key difference between the HTTP PUT and PATCH methods?

- (A) PUT and PATCH behave identically in every case
- (B) PUT typically replaces/updates a resource entirely, while PATCH applies a partial update to a resource
- (C) PUT is used only for deleting resources
- (D) PATCH can only be used with XML payloads, unlike PUT

Correct Answer: (B)

**Explanation:** Conventionally, `PUT` sends a full representation of the resource to replace the existing one, while `PATCH` sends only the fields that need to change, leaving the rest untouched.

Git / GitHub Medium

C12.

What is the difference between `git fetch` and `git pull`?

- (A) `git fetch` downloads changes from the remote without merging them into your local branch; `git pull` does a fetch followed by a merge (or rebase) into your current branch
- (B) `git fetch` automatically merges changes; `git pull` only downloads without merging
- (C) Both commands do exactly the same thing
- (D) `git fetch` pushes local commits up to the remote repository

Correct Answer: (A)

**Explanation:** `git fetch` is the "safe" option — it updates your local copy of the remote's history without touching your working branch. `git pull` = `git fetch` + `git merge` (or rebase, depending on configuration) into your current branch.

C13.

**What is the difference between an INNER JOIN and a LEFT JOIN in SQL?**

- (A) INNER JOIN returns all rows from the left table regardless of a match; LEFT JOIN returns only matching rows
- (B) INNER JOIN returns only rows with matches in both tables; LEFT JOIN returns all rows from the left table plus matched rows from the right table (NULL where there's no match)
- (C) INNER JOIN and LEFT JOIN always produce identical results
- (D) LEFT JOIN requires both tables to have identical column names

**Correct Answer: (B)**

**Explanation:** INNER JOIN excludes unmatched rows from both sides. LEFT JOIN keeps every row from the left (first) table, filling in NULLs for any columns from the right table where no match exists.

C14.

**Which SQL query correctly returns the second-highest salary from an Employee table?**

- (A) SELECT salary FROM Employee ORDER BY salary ASC LIMIT 1;
- (B) SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);
- (C) SELECT salary FROM Employee WHERE salary = (SELECT MAX(salary) FROM Employee);
- (D) SELECT AVG(salary) FROM Employee;

**Correct Answer: (B)**

**Explanation:** Option (A) returns the lowest salary, not the second-highest. Option (C) returns the highest salary itself. Option (D) returns the average. Option (B) correctly excludes the overall maximum first, then finds the max of what remains — giving the second-highest value.

C15.

**What is CORS (Cross-Origin Resource Sharing)?**

- (A) A browser security feature that restricts web pages from making requests to a different domain/origin than the one that served the page, unless the server explicitly allows it via response headers
- (B) A JavaScript library used for making HTTP requests
- (C) A CSS property used for cross-browser styling consistency
- (D) A database replication technique

**Correct Answer: (A)**

**Explanation:** CORS is enforced by the browser, not the server. When a frontend (e.g., running on localhost:3000) calls an API on a different origin, the browser blocks the response unless the server includes headers like `Access-Control-Allow-Origin` permitting that origin — a very common thing to configure when a React app talks to a separate backend API.

## Section 2 · React JS (15 Questions)

Focused on core React concepts a React JS Developer should know — components, JSX, props/state, hooks, routing, context, and performance basics.

Components Easy

R1.

**What is the difference between functional and class components in React?**

- (A) Functional components cannot have state or lifecycle behavior at all
- (B) Class components use ES6 class syntax with lifecycle methods and `this.state`; functional components are plain JavaScript functions that, with Hooks (since React 16.8), can also manage state and side effects, and are now the more common approach
- (C) Functional and class components can never exist in the same application
- (D) Class components always render faster than functional components

**Correct Answer: (B)**

**Explanation:** Before Hooks, only class components could hold state and respond to lifecycle events. Hooks (`useState`, `useEffect`, etc.) gave functional components the same capabilities with simpler syntax, which is why most modern React code is written as functional components.

JSX Easy

R2.

**What is JSX?**

- (A) A templating language that only works in Node.js, not in the browser
- (B) A syntax extension to JavaScript that looks like HTML and is compiled (typically via Babel) into `React.createElement()` calls
- (C) A feature that replaces the need for JavaScript entirely in a React app
- (D) A syntax that can only be used inside class components

**Correct Answer: (B)**

**Explanation:** JSX isn't valid JavaScript by itself — a build tool like Babel transforms `<div>Hello</div>` into `React.createElement('div', null, 'Hello')` before the browser ever sees it.

Props & State Easy

R3.

**What is the difference between props and state in React?**

- (A) Props are mutable and owned by the child component; state is immutable and owned by the parent
- (B) Props are read-only data passed from a parent component to a child; state is local, mutable data managed within a component
- (C) Props and state are two names for the same concept
- (D) A child component can directly reassign a prop's value to change it

**Correct Answer: (B)**

**Explanation:** Props flow one-way, from parent to child, and the child must treat them as read-only. State belongs to the component itself and can be updated (via `useState` or `this.setState`), triggering a re-render.

Hooks · useState Easy

R4.

**What does the `useState` hook return?**

- (A) A single value representing the current state, with no way to update it
- (B) An array with the current state value and a setter function to update it, e.g. `const [count, setCount] = useState(0)`
- (C) An object that can only store numeric values
- (D) A value that is automatically persisted to `localStorage`

**Correct Answer: (B)**

**Explanation:** `useState` returns a two-element array: the current value and a function to update it. Calling the setter schedules a re-render with the new value, and state is reset on unmount (it is not automatically persisted anywhere).

## R5.

In `useEffect(() => { ... }, [count])`, when does the effect run?

- (A) On every single render, regardless of what changed
- (B) Only once, when the component first mounts, and never again
- (C) On mount, and again on any re-render where the value of `count` has changed since the last render
- (D) Only when the component is about to unmount

**Correct Answer: (C)**

**Explanation:** The dependency array controls when the effect re-runs. `[count]` means: run after the first render, then again only on renders where `count`'s value differs from the previous render. (An empty array `[]` would mean "mount only"; omitting the array entirely means "every render.")

## R6.

How does event handling in React typically differ from vanilla JavaScript DOM event handling?

- (A) React wraps native DOM events in a cross-browser wrapper called a `SyntheticEvent`, and handlers are attached as camelCase JSX props like `onClick` instead of calling `addEventListener` directly
- (B) React does not support event handling at all; `addEventListener` must always be used manually
- (C) Event handlers in React must be defined in separate `.css` files
- (D) React uses the exact same lowercase attribute names as plain HTML, like `onclick` and `onchange`

**Correct Answer: (A)**

**Explanation:** Instead of `element.addEventListener('click', fn)`, React lets you write `<button onClick={handleClick}>` directly in JSX. Internally, React normalizes events across browsers using `SyntheticEvent` so your handlers behave consistently.

## R7.

Which of the following is a valid way to conditionally render content in JSX?

- (A) Conditional rendering is not possible in React; every element must always render and be hidden with CSS instead
- (B) Using a ternary operator (`condition ? <A /> : <B />`) or a logical `&&` (`condition && <A />`) directly inside JSX
- (C) Only a switch statement can be used, and it must be written inline inside JSX
- (D) Conditional rendering requires installing a separate npm package

**Correct Answer: (B)**

**Explanation:** Since JSX is just JavaScript, you can use any expression that evaluates to a renderable value — ternaries and `&&` are the most common patterns, alongside early returns or `if/else` logic written above the return statement.

## R8.

Why does React ask for a unique "key" prop when rendering a list, and what can go wrong if you use the array index as the key?

- (A) Keys are purely optional and have no effect on how React renders a list
- (B) A unique, stable key helps React identify which items changed, were added, or removed between renders; using the array index as a key can cause bugs (wrong items re-rendering, lost input state) if the list is reordered, filtered, or has items inserted/removed
- (C) Keys must always be the array index for React to render correctly
- (D) Keys are only required when a list has more than 100 items

**Correct Answer: (B)**

**Explanation:** React uses keys to match list items between renders during reconciliation. If the index is used as a key and the list order changes, React can mismatch items to the wrong DOM nodes, causing stale state or incorrect rendering — a stable unique ID (like a database id) should be used instead.

## R9.

**What is the difference between a controlled and an uncontrolled component in a React form?**

- (A) In a controlled component, the form data is stored in React state and updated via an onChange handler; in an uncontrolled component, the DOM itself manages the value, typically read via a ref when needed
- (B) Controlled components cannot use onChange handlers at all
- (C) Uncontrolled components automatically stay in sync with React state with no extra code required
- (D) There is no real difference between the two; the terms are interchangeable

**Correct Answer: (A)**

**Explanation:** Controlled inputs give React full control (and make validation/formatting easier) because every keystroke updates state. Uncontrolled inputs let the DOM hold the value directly, and you pull the value out only when needed (e.g., on submit) via a ref — useful for simple forms or integrating with non-React code.

## R10.

**What is the standard pattern for fetching data from an API in a React functional component?**

- (A) Call fetch() directly in the component's render body, with no hook involved
- (B) Use useEffect with an appropriate dependency array to trigger the fetch (commonly once on mount with []), storing the result in state via useState so the component re-renders with the data
- (C) Fetching data from an API is not possible inside functional components
- (D) API calls must always be made from a class component's componentDidMount

**Correct Answer: (B)****Explanation:**

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/users")
      .then(res => res.json())
      .then(data => { setUsers(data); setLoading(false); });
  }, []); // empty array = run once, on mount

  if (loading) return <p>Loading...</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Calling fetch directly in the render body (option A) would trigger a new request on every render, causing an infinite loop once state is updated.

## R11.

**What is the difference between React Router's <Link> component and a normal <a> tag?**

- (A) <Link> and <a> behave identically; there's no practical difference
- (B) <Link> performs client-side navigation without a full page reload, preserving the single-page app's in-memory state; a normal <a> tag triggers a full browser page reload
- (C) <a> tags cannot be used anywhere inside a React application
- (D) <Link> only works inside class components

**Correct Answer: (B)**

**Explanation:** React Router intercepts the navigation and swaps the rendered route component using the History API, instead of letting the browser reload the whole page and re-download all assets — which is the main performance benefit of a single-page app.

**R12.****What problem does the Context API solve?**

- (A) It is used only for applying global CSS styles
- (B) It lets you share data across the component tree without manually passing props down through every intermediate level ("prop drilling")
- (C) It completely replaces the need for any local component state
- (D) It only works if Redux is also installed in the project

**Correct Answer: (B)**

**Explanation:** Without Context, data like a logged-in user or theme setting would need to be passed as props through every component in between, even ones that don't use it. Context lets any descendant component read the value directly via `useContext`, skipping the intermediate levels.

**R13.****How does `useEffect` in a functional component relate to the class lifecycle methods `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`?**

- (A) These class lifecycle behaviors have no equivalent in functional components
- (B) A `useEffect` with an empty dependency array (`[]`) behaves like `componentDidMount`; a `useEffect` with specific dependencies also behaves like `componentDidUpdate` for those dependencies; and the cleanup function returned from an effect behaves like `componentWillUnmount`
- (C) Lifecycle concepts only apply to class components; functional components never mount or unmount
- (D) `componentWillUnmount` always runs before `componentDidMount` in the same component

**Correct Answer: (B)****Explanation:**

```
useEffect(() => {
  console.log("mounted / dependency changed"); // ~ componentDidMount / componentDidUpdate
  return () => {
    console.log("cleanup before unmount or re-run"); // ~ componentWillUnmount
  };
}, [someValue]);
```

`useEffect` unifies these three lifecycle concerns into one hook, with the dependency array controlling when it re-runs and the returned function handling cleanup.

**R14.****What is the Virtual DOM, and how does React use it?**

- (A) The Virtual DOM is an exact, always-synced copy of the real DOM that updates at the same instant as the real DOM
- (B) The Virtual DOM is a lightweight, in-memory representation of the UI; React compares ("diffs") it against the previous version and applies only the minimal necessary changes to the real DOM, improving performance
- (C) The Virtual DOM completely eliminates the need for a real DOM in the browser
- (D) The Virtual DOM is a feature available only in class components

**Correct Answer: (B)**

**Explanation:** Directly manipulating the real DOM for every small change is expensive. React updates its lightweight Virtual DOM first, diffs it against the prior snapshot (reconciliation), and then batches and applies only the actual changes needed to the real DOM.

R15.

**What is the purpose of the useMemo and useCallback hooks?**

- (A) useMemo memoizes a computed value, and useCallback memoizes a function reference — both help avoid expensive recalculations or unnecessary re-renders of child components that rely on referential equality (e.g., components wrapped in React.memo)
- (B) useMemo and useCallback only work inside class components
- (C) useMemo is used specifically to fetch data from an API
- (D) useCallback guarantees that a component will never re-render again

**Correct Answer: (A)**

**Explanation:** Without memoization, an expensive calculation re-runs on every render, and a new function reference is created on every render (which can cause a React.memo-wrapped child to re-render unnecessarily, since it looks like a "new" prop). useMemo caches a computed value and useCallback caches a function reference, both recomputed only when their dependencies change.